

Zed AI 3大プロテクニック ― 明日からの自走を劇的に変えるインプット

本日最後のインプット。なぜこのスキルが必要なのか、そして明日からの自走を劇的に変える「Zed AIの3大プロテクニック」を構造化して解説する。

なぜ「ZedのAIを使い倒すスキル」が必須なのか

これまでの開発では、エラー文をコピーし、ブラウザや別のチャット画面に貼り付けてAIに修正させていた。この「往復」が、実は最大の時間泥棒だった。

プロのエンジニアはこの往復をしない。**「エディタ(Zed)にコードを直接見せながら、その場でAIに書き換えさせる」**という、次元の違うスピードで開発を進めている。

これを身につけると、今後どんなエラーに直面しても、他人に頼らず自分の手だけでバグを解決し、自走できるようになる。エディタの外に一度も出ずに「見せる・直す・戻す」が完結する——これが到達点だ。

明日から使える Zed内蔵AI (Assistant Panel) の3大プロ技

Zedの画面左側のチャット欄 (Assistant Panel) で、今すぐ使える強力な機能。

プロ技1: `/file` コマンドによる「コードの超高速・指差し確認」

チャット入力欄に `/file` と打つと、フォルダ内のファイル一覧がポップアップする。そこで対象ファイル (例: `ai_app.py`) を選ぶと、チャット欄の中にプログラムコードが丸ごと一瞬で読み込まれる。

効果: わざわざコードをコピー & ペーストしてAIに見せる必要がなくなる。

None

`/file ai_app.py` このコードの90行目を解説して

こう指示するだけで、AIがコード全体を理解したうえでピンポイントに解説してくれる。「一部だけ貼って文脈が足りずトンチンカンな回答が返る」という事故が消える。

 プロ技2: `cmd + enter` による「コードの自動書き換え(インライン生成)」
左側のチャット欄でAIが新しいコードを提案してきたら、コードブロック右上の「Insert」(または矢印マーク)をクリックする。

効果: マウスでのコピー & ペーストの手間すら省き、AIが作ったコードを右側の `ai_app.py` へ1秒で自動的に流し込んで上書きしてくれる。コピペミスや、貼り付け時にコードがちぎれるエラーが物理的にゼロになる。

これまで `git remote add` のURLがコピペで欠けて何十分も溶かした——あの類のミスが、構造的に起きなくなる。手作業の転記をなくすことが、そのままバグの根絶につながる。

プロ技3: コンテキスト(文脈)の完全リセット

チャットが長くなると、ローカルAI(Qwen3.5)は過去の会話をすべて読み直すため、最初の**「タメ(沈黙)」がどんどん長くなる**。

効果: チャット画面上部の「New Thread」(プラスマークのアイコン) をカチッと押すと、過去の記憶が綺麗にリセットされ、次の質問への初速が一瞬(タメがゼロ)に復活する。

これは開発ログで実測した現象と直結する。大きなコンテキスト長のモデルは、履歴が伸びるほど毎回すべてを再スキャンして初速が雪だるま式に遅くなる。話題が切り替わったら `New Thread` で切る——これが軽快さを保つ最大のコツ。

3大技の使い分け(どの技を、いつ)

場面	使う技	何が起きるか
コードの意味・バグを聞きたい	プロ技1(<code>/file</code>)	コピペ不要でAIに全体を見せて質問
AIの提案を反映したい	プロ技2(Insert)	転記ミスゼロで自動上書き
応答の初速が重くなってきた	プロ技3(New Thread)	文脈リセットでタメを解消

明日からの実践フロー(この3つを1本につなぐ)

1. 見せる — `/file ai_app.py` でコードをAIに丸ごと渡し、「このエラーを直して」と頼む(プロ技1)
2. 直す — 提案されたコードを **Insert** でその場で上書き反映する(プロ技2)
3. 切る — 別の話題・別のバグに移るときは **New Thread** で文脈をリセットし、初速を取り戻す(プロ技3)

この「見せる → 直す → 切る」のループを、エディタの外に一度も出ずに回す。ブラウザや別チャットへの往復が消え、バグ解決が体感で数倍速くなる。これが、明日からの自走を支える中核スキルになる。

補足メモ

- ショートカット(`cmd + enter` など)や各アイコンの位置・名称は、Zedのバージョン更新で変わることがある。見当たらないときはコマンドパレット(`cmd + shift + p` / Windowsは `Ctrl + Shift + P`)で「file」「insert」「new thread」等を検索すれば同じ操作にたどり着ける。
- 参照モデルは、初速重視なら軽量モデル(例: `qwen2.5-coder:7b`)、精度重視なら現行モデル+RAG。プロ技3で文脈を切ることと、軽量モデルを選ぶことは、どちらも「タメを消す」ための両輪。
- Windowsで作業する場合は `cmd` を `Ctrl` に読み替える(引き継ぎ手順書の対応表を参照)。